Facultad de Defensa Nacional

Licenciatura en CIBERDEFENSA

Lenguajes de Programación

Introducción a la Programación

Guía Práctica 5

Docentes:

Ing. Sebastián Balestrini

Esp. Ing. Gustavo Di Stefano

Carga horaria: 64 hs

Guía Práctica 4: Funciones y modularización

Instrucciones generales:

- Utilizar el manual de referencia del Lenguaje C de la cátedra.
- Utilizar https://www.onlinegdb.com/ para codificar en C
- Antes de empezar a codificar, chequear que los tipos de datos seleccionados para las variables se ajusten al uso del problema.

Ejercicios :

Básicos (Nivel básico)

1. Función suma y resta

   Escribir un programa que lea dos enteros y muestre su suma y resta utilizando dos funciones separadas.

2. Cuadrado y cubo

   Implementar dos funciones: una que calcule el cuadrado de un número y otra que calcule el cubo. Pedir un número y mostrar ambos resultados.

3. Mayor de dos números

   Escribir una función que reciba dos números enteros y retorne el mayor de ellos. Probar la función en main.

4. Imprimir cartel (procedimiento)

   Crear un procedimiento que reciba un número y muestre si es positivo, negativo o nulo (sin return, solo printf).

5. Factorial iterativo

   Implementar una función que calcule el factorial de un número de manera iterativa. (no recursiva)

Problemas Avanzados (Nivel intermedio – avanzado)

6. Factorial recursivo

   Reescribir el problema 5 usando recursividad.

7. Parámetros por referencia

   Implementar un procedimiento intercambiar(&a, &b) que reciba dos enteros por punteros y los intercambie. Probar en main.

8. Operaciones con punteros

   Escribir una función que reciba dos enteros por punteros y devuelva (por parámetros de salida) su suma, resta, producto y cociente.

9. Mini-calculadora modularizada

   Con menú: sumar, restar, multiplicar, dividir y salir. Cada operación debe ser una función. Validar división por cero.

10. Vector y promedio

    Generar un vector de N=20 números aleatorios entre 0 y 10. Hacer una función que calcule el promedio y un procedimiento que muestre el vector.

11. Vector binario invertido

    Ingresar un vector de 10 números binarios (0 y 1). Crear otro vector con los valores invertidos usando un procedimiento.

12. Conteo pares e impares con punteros

    Escribir una función que reciba un vector de enteros y dos punteros, uno para almacenar la cantidad de pares y otro la cantidad de impares.

13. Hipotenusa y perímetro

    Implementar funciones que calculen la hipotenusa y el perímetro de un triángulo rectángulo (usar math.h).

14. Matriz y sumatoria

    Llenar una matriz 4x4 con números al azar del 1 al 6 (simulando dados). Hacer una función que devuelva la suma de las filas impares.

15. Matriz traspuesta

    Crear una matriz 3x4 con números aleatorios entre 0 y 9. Implementar un procedimiento que la traspuesta y la muestre.

16. Menú sobre matrices binarias

    Crear una matriz de 5x5 con números binarios al azar. Hacer un menú con opciones: mostrar matriz, mostrar traspuesta, invertir bits (0↔1), generar nueva matriz, salir.

17. Promedio de vectores

    Ingresar 3 vectores de 20 enteros. Crear otro vector con el promedio de cada posición (usar float). Mostrar el vector resultante.

18. Vector pares a 0 e impares a 1

    Generar un vector de 20 números enteros al azar (1 al 10). Crear un procedimiento que reemplace pares por 0 e impares por 1. Mostrar resultado y cantidad de pares/impares.

19. Máximo y mínimo en un vector

    Generar un vector de 30 enteros al azar (1 al 100). Implementar una función que devuelva el máximo y el mínimo usando parámetros por referencia.

20. Serie recursiva (sumatoria)

    Implementar una función recursiva que calcule la sumatoria de un número n (ej: si n=5, devuelve 5+4+3+2+1=15).

Números Aleatorios - Como generar

Aclaraciones sobre **rand()** y **srand()**

- En C, la función rand() genera números enteros pseudoaleatorios entre 0 y un valor grande (RAND_MAX).
- Para limitar el rango se usa el operador módulo %.

  Ejemplo: rand() % 11 → genera valores de 0 a 10.
- Para que los valores no sean siempre los mismos cada vez que ejecutes el programa, se usa **srand(time(NULL))** al inicio (necesita #include <time.h>).

En el siguiente ejemplo, se muestra el uso de la función **rand()**

```c
#include <stdio.h>
#include <stdlib.h> // rand(), srand()
#include <time.h>   // time()
#define N 10    // cantidad de numeros a generar


int main() {

    int i;

    // Inicializar semilla para números aleatorios
    srand(time(NULL));

    // Generar números aleatorios entre 0 y 10
    for (i = 0; i < N; i++) {
        printf("%d ", rand()%11);

    }


    //salto de linea
    printf("\n");
    return 0;
}
```